



LINGI2251 : Software Quality Assurance Answers



Année académique 2018 - 2019

Enseignant : CHARLES Pecheur

Cours : LINGI2251

Collaborateurs : Crochet Christophe, Old
authors'synthesis

Lien : <https://www.overleaf.com/4685621966pzdpvkrkszhq>

Table des matières

1	Software Quality	1
1.1	Define aspects of software defects and defect management alternatives. Discuss different software quality characteristics and perspectives. Define correctness, reliability, safety and robustness.	1
1.1.1	Define aspects of software defects and defect management alternatives	1
1.1.2	Software Quality : What Is Good Software?	1
1.1.3	Define correctness, reliability, safety and robustness.	2
2	Software development process	3
2.1	Define software verification and validation. Describe a software development process and how verification and validation activities fit into this process. Describe different types of software quality assurance activities.	3
2.1.1	Define software verification and validation.	3
2.1.2	Describe a software development process and how verification and validation activities fit into this process.	3
2.1.3	Describe different types of software quality assurance activities.	3
3	Behaviour models	5
3.1	Define state models. Describe control flow graphs and their constituents. Describe call graphs and discuss context-sensitive analysis. Describe finite state machines and discuss abstraction.	5
3.1.1	Define state models.	5
3.1.2	Describe control flow graphs and their constituents.	5
3.1.3	Describe call graphs and discuss context-sensitive analysis.	6
3.1.4	Describe finite state machines and discuss abstraction.	6
4	Data models	7
4.1	Define data dependence based on def-use pairs. Describe data dependence and control dependence graphs. Explain the general principle of dataflow analyses based on worklist algorithms, and the particular case of computing reaching definitions. Discuss the effect of pointers and procedures.	7
4.1.1	Define data dependence based on def-use pairs.	7
4.1.2	Describe data dependence and control dependence graphs.	7
4.1.3	Explain the general principle of dataflow analyses based on worklist algorithms, and the particular case of computing reaching definitions.	8
4.1.4	Discuss the effect of pointers and procedures.	9
5	Functional testing	11
5.1	Define test case, test obligation, adequacy criterion, test satisfaction. Define functional and structural testing. Explain category-partition testing. Explain pair-wise and n-wise testing. Explain catalog-based testing.	11
5.1.1	Define test case, test obligation, adequacy criterion, test satisfaction.	11
5.1.2	Define functional and structural testing.	11
5.1.3	Explain category-partition testing.	11
5.1.4	Explain pair-wise and n-wise testing.	12
5.1.5	Explain catalog-based testing.	12

6	Structural testing	14
6.1	Define statement, branch and condition coverage, compound conditions and MC/DC, and compare their strengths. Define path coverage, discuss limitations and show some practical path coverage criteria. Define data flow coverage criteria. Discuss the problems of aliases and infeasibility.	14
6.1.1	Define statement, branch and condition coverage, compound conditions and MC/DC, and compare their strengths.	14
6.1.2	Define path coverage, discuss limitations and show some practical path coverage criteria.	15
6.1.3	Define data flow coverage criteria.	15
6.1.4	Discuss the problems of aliases and infeasibility.	15
7	Model-based testing	16
7.1	Explain the principles of model-based testing. Explain path-insensitive and path-sensitive state machine coverage criteria. Discuss coverage criteria for decision structures and grammars.	16
7.1.1	Explain the principles of model-based testing.	16
7.1.2	Explain path-insensitive and path-sensitive state machine coverage criteria.	16
7.1.3	Discuss coverage criteria for decision structures and grammars.	16
8	Object-oriented testing	17
8.1	Explain the principles of testing of object-oriented software, at the unit and integration level. Discuss structural coverage criteria for intra- and inter-class testing. Discuss the issues of test oracles, polymorphism and exception handling.	17
8.1.1	Explain the principles of testing of object-oriented software, at the unit and integration level.	17
8.1.2	Discuss structural coverage criteria for intra- and inter-class testing.	17
8.1.3	Discuss the issues of test oracles, polymorphism and exception handling.	18
9	Fault-based testing	19
9.1	Explain the principles and assumptions of mutation testing. Give examples of mutation operators. Discuss fault-based coverage measures. Discuss ways to reduce the cost of mutation testing. explain fault estimation using seeded faults or independent test groups.	19
9.1.1	Explain the principles and assumptions of mutation testing.	19
9.1.2	Give examples of mutation operators.	19
9.1.3	Discuss fault-based coverage measures.	19
9.1.4	Discuss ways to reduce the cost of mutation testing.	20
9.1.5	Explain fault estimation using seeded faults or independent test groups.	20
10	Test execution	21
10.1	Describe the principles of scaffolding for test execution. Discuss different types of test oracles. Describe the nature and objectives of unit, integration, system, acceptance and regression testing activities. Discuss different integration testing strategies. Explain regression test selection and prioritization.	21
10.1.1	Describe the principles of scaffolding for test execution.	21
10.1.2	Discuss different types of test oracles.	22
10.1.3	Describe the nature and objectives of unit, integration, system, acceptance and regression testing activities.	23
10.1.4	Discuss different integration testing strategies.	24
10.1.5	Explain regression test selection and prioritization.	24

11 Program analysis = Execute tests	25
11.1 Describe principles of program inspection. Define static and dynamic program analysis. Describe the principles of symbolic program execution. Describe the principles of program verification using symbolic execution. Discuss contract-based reasoning on procedures and data structures. Describe the principles of symbolic testing and dynamic program analyses. Discuss lockset analysis.	25
11.1.1 Describe principles of program inspection.	25
11.1.2 Define static and dynamic program analysis.	25
11.1.3 Describe the principles of symbolic program execution.	25
11.1.4 Describe the principles of program verification using symbolic execution.	26
11.1.5 Discuss contract-based reasoning on procedures and data structures.	26
11.1.6 Describe the principles of symbolic testing and dynamic program analyses.	27
11.1.7 Discuss lockset analysis.	27
12 Finite state analysis	28
12.1 Describe the principles of finite-state analysis. Define safety and liveness properties and discuss their verification. Discuss the model size and correspondence problems. Discuss intensional representations. Discuss iterative model refinement. Discuss data model verification.	28
12.1.1 Describe the principles of finite-state analysis.	28
12.1.2 Define safety and liveness properties and discuss their verification.	28
12.1.3 Discuss the model size and correspondence problems.	29
12.1.4 Discuss intensional representations.	29
12.1.5 Discuss iterative model refinement.	29
12.1.6 Discuss data model verification.	30
13 Software measurement : size	31
13.1 State the general principles of software measurement. Discuss size measurement metrics based on lines of code. Explain functional size measurement using function points.	31
13.1.1 State the general principles of software measurement.	31
13.1.2 Discuss size measurement metrics based on lines of code.	31
13.1.3 Explain functional size measurement using function points.	32
14 Software measurement : structure	33
14.1 Describe the principles of structure measurement. Describe hierarchical measures based on prime decomposition of control flow graphs. Define cyclomatic complexity measure. Discuss design-level, inter-modular complexity measures.	33
14.1.1 Describe the principles of structure measurement.	33
14.1.2 Describe hierarchical measures based on prime decomposition of control flow graphs.	33
14.1.3 Define cyclomatic complexity measure.	34
14.1.4 Discuss design-level, inter-modular complexity measures.	34
15 Software measurement : quality	35
15.1 Describe the principles and elements of quality models. Discuss defect-based quality measures. Discuss usability, maintainability and security measures.	35
15.1.1 Describe the principles and elements of quality models	35
15.1.2 Discuss defect-based quality measures.	36
15.1.3 Discuss usability, maintainability and security measures.	37

16 Software reliability	40
16.1 Describe the model of reliability based on failure rates. Define failure probability functions, reliability, maintainability, availability, MTTF and MTTR. Describe the case of constant failure rates. Describe the principles of failure prediction systems. Discuss the example of the Jelinski-Moranda model. Discuss statistical testing.	40
16.1.1 Describe the model of reliability based on failure rates.	40
16.1.2 Define failure probability functions, reliability, maintainability, availability, MTTF and MTTR.	40
16.1.3 Describe the case of constant failure rates.	40
16.1.4 Describe the principles of failure prediction systems.	41
16.1.5 Discuss the example of the Jelinski-Moranda model.	41
16.1.6 Discuss statistical testing.	41

1 Software Quality

1.1 Define aspects of software defects and defect management alternatives. Discuss different software quality characteristics and perspectives. Define correctness, reliability, safety and robustness.

1.1.1 Define aspects of software defects and defect management alternatives

Defect = some problem with the software (Human error → Fault → Failure)

- (a) = **Error** : a mistake in performing some software activities
- (b) = **Fault** : a defect in the product
- (c) = **Failure** : a departure from the system's required behaviour

⇒ *High quality* ≈ low defect

⇒ *Quality problem* ≈ defect impact

Dealing with defects :

1. Defect prevention :

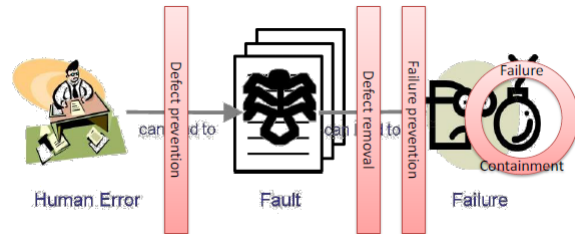
- Prevent faults from being injected
- Error blocking, error source removal

2. Defect removal :

- Remove faults
- Inspection (find faults), testing (find failures from faults)

3. Defect containment :

- Keep failures local, reduce failure impact
- Fault-tolerance, failure containment



1.1.2 Software Quality : What Is Good Software ?

1. **Transcendental view** : Quality is an ideal that we thrive to but cannot attain
2. **User view** : Quality is fitness for purpose, reliability, absence of defects
3. **Manufacturing view** : Quality is conformance to the process
4. **Product view** : Quality is showing good inherent characteristics
5. **Value-based view** : Quality is how much the customer is willing to pay for it

Quality models relate the user's external view to the developer's internal view :

1. Consumers =

- (a) **Client** : pay for the development
- (b) **User** of software
- (c) **Customers** : buy after development

Expect **external** qualities :

- (a) good enough for the price
- (b) fit-for-use, doing the right things
- (c) conformance, doing things right

⇒ **Judge external characteristics** : number and type of **failures**

2. Producers = Developer

Expect **internal** qualities :

- (a) good enough for the cost
- (b) maintainable
- (c) interoperable
- (d) modular

⇒ **Judge internal characteristics** : number and **fault**

Quality characteristics (for customers) :

1. **Functionality** :

A set of attributes that bears on the existence of a set of functions and their specified properties. The functions are those that satisfy stated or implied needs

2. **Reliability** :

A set of attributes that bears on the capability of software to maintain its performance level under stated conditions for a stated period of time

3. **Usability** :

A set of attributes that bears on the effort needed for use and on the individual assessment of such use by a stated or implied set of users

4. **Efficiency** :

A set of attributes that bears on the relationship between the software performance and the amount of resources used under stated conditions

5. **Maintainability** :

A set of attributes that bears on the effort needed to make specified modifications (which may include corrections, improvements, or adaptations of software to environmental changes and changes in the requirements and functional specifications)

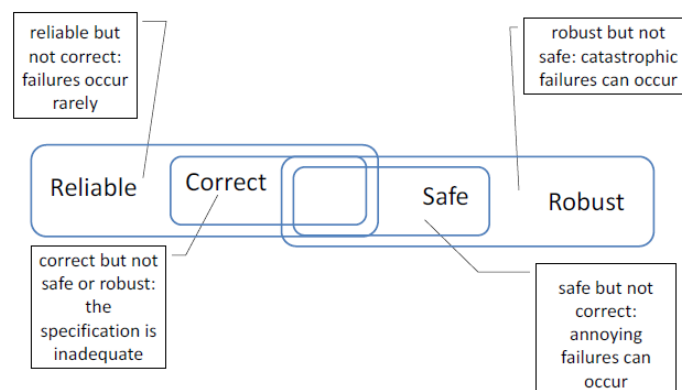
6. **Portability** :

A set of attributes that bears on the ability of software to be transferred from one environment to another (including the organizational, hardware, or software environment)

1.1.3 Define correctness, reliability, safety and robustness.

These are "dependability properties" (Correctness properties are **undecidable**) :

- **Correctness** : A program is correct if it is consistent with its specification (seldom practical for non-trivial systems)
- **Reliability** : likelihood of correct function for some "unit" of behavior, relative to a specification and usage profile, statistical approximation to correctness (100% reliable = correct)
- **Safety** : preventing hazards
- **Robustness** : acceptable (degraded) behavior under extreme conditions



2 Software development process

2.1 Define software verification and validation. Describe a software development process and how verification and validation activities fit into this process. Describe different types of software quality assurance activities.

2.1.1 Define software verification and validation.

1. **Validation** : = does the software system meets the user's real needs? are we building the right software?

⇒ specifications accurately reflects the customer's needs

2. **Verification** : = does the software system meets the requirements specifications? are we building the software right?

⇒ application conform to specifications

Techniques :

- (a) Modeling
- (b) Scenarios
- (c) Prototypes
- (d) Simulation
- (e) ...

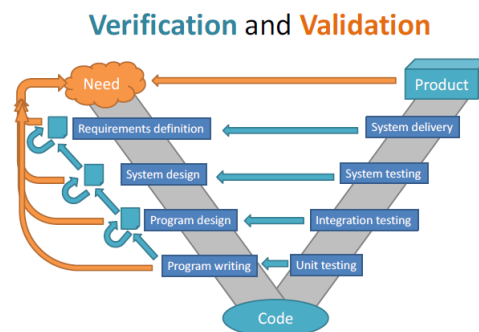
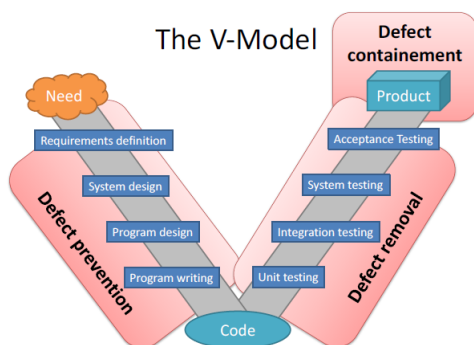
Techniques :

- (a) Consistency/Completeness/Reachability checks
- (b) Model checking
- (c) Mathematical proofs

2.1.2 Describe a software development process and how verification and validation activities fit into this process.

Classic approach : Requirement ⇒ Specification ⇒ Design ⇒ Coding ⇒ Testing ⇒ Release

Variation : waterfall, iterative, spiral, agile, XP, ...



2.1.3 Describe different types of software quality assurance activities.

1. Testing :

- Executed late in development
- Start as early as possible
- Early test generation has several advantages
 - (a) Tests generated independently from code, when the specifications are fresh in the mind of analysts
 - (b) The generation of test cases may highlight inconsistencies and incompleteness of the corresponding specifications
 - (c) tests may be used as compendium of the specifications by the programmers

2. Inspection :

- can be applied to essentially any document
 - (a) requirements statements
 - (b) architectural and detailed design documents
 - (c) test plans and test cases
 - (d) program source code
- may also have secondary benefits
 - (a) spreading good practices
 - (b) instilling shared standards of quality
- takes a considerable amount of time
- re-inspecting a changed component can be expensive
- used primarily
 - (a) where other techniques are inapplicable
 - (b) where other techniques do not provide sufficient coverage

3. Automatic Static Analysis :

- More limited in applicability
 - (a) can be applied to some formal representations of requirements models
 - (b) not to natural language documents
- are selected when available
 - (a) substituting machine cycles for human effort makes them particularly cost-effective

4. Computer-Aided Verification :

- **Model checking** : exhaustive search of a specification's execution space
 - (a) applicable to behavior models (e.g. statecharts, Petri nets)
 - (b) check state conditions, temporal logic, compare models
- **Theorem proving** : prove Specifications AND Assumptions IMPLY Requirements
 - (a) using built-in theories, inference rules, decision procedures

5. Improving the Process :

- Long lasting errors are common
- It is important to structure the process for
 - (a) the most critical persistent faults
 - (b) tracking them to frequent errors
 - (c) adjusting the development and quality processes to eliminate errors
- Feedback mechanisms are the main ingredient of the quality process for identifying and removing errors

3 Behaviour models

3.1 Define state models. Describe control flow graphs and their constituents. Describe call graphs and discuss context-sensitive analysis. Describe finite state machines and discuss abstraction.

3.1.1 Define state models.

- **Models** = **abstraction** of the system (removes irrelevant attributes or details via an abstraction function)
 - Represent a system, and artifact, a design
 - analyse a system, an artifact, a design
 - before the system is build
 - easier to analyse/check/test than the actual system

Properties : Compact, Predictive, Semantically meaningful and Sufficiently general

• State models

Program execution = sequence of states and transitions

- **States** : control + data (Location + variables, stack, heap)
- **Transitions** : actions (ops, instructions)

State Space :

- *Full* (all possible values)
- *Reachable* (from initial states)

Essentially **infinite**.

Finite models of program execution

⇒ abstraction

1. Execution is **coarsened** (fewer steps)
2. **Nondeterminism** is introduced

3.1.2 Describe control flow graphs and their constituents.

Abstraction : set of program locations (PC) → finite number of locations

Control Flow Graph (CFG) :

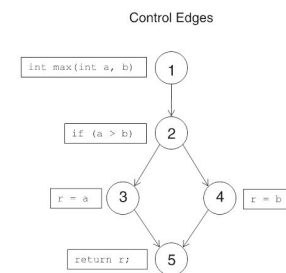
- **Nodes** = regions of source code (basics blocks)
 - Basic block : maximal program region with a single entry and single exit point
 1. Often several statements in one blocks
 2. Sometimes one statement in several blocks
- **Edge** = possibility that execution proceeds from the end of one region to the beginning of another
 - **Linear Code Sequence and Jump (LCSJ)** :
Subpaths from one branching point to another (jumps) proceeding to next block is not branching

Control Flow Graph

```

1 int main(int a, b) {
2   int r;
3   if (a > b)
4     r = a;
5   else
6     r = b;
7   return r;
8 }

```



⇒ **Intra-procedural** (ignore calls). May not cover some flows (e.g : exceptions are not covered)

3.1.3 Describe call graphs and discuss context-sensitive analysis.

Call graphs :

- Nodes represent procedures
- Edges represent calls relation

⇒ **Inter-procedural**

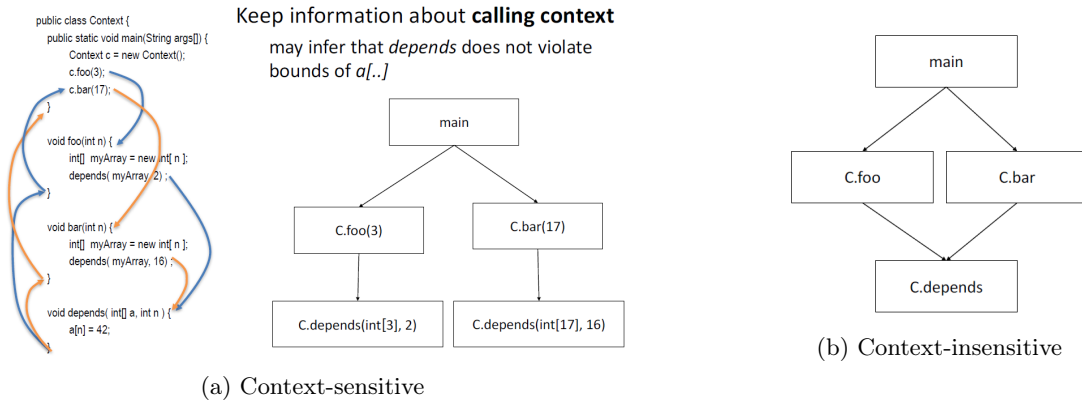


FIGURE 1 – VS

Context-sensitive analysis : the number of contexts grows exponentially with the depth of the calls.

- $\#C \approx \#P^{\text{depth}}$: where $\#C$ = number of contexts and $\#P$ = number of procedures

3.1.4 Describe finite state machines and discuss abstraction.

- **Nodes** = states (finite number)
- **Edges** = transitions
 - labelled with condition, operation, even
 - input/output : Mealy Machine

⇒ Used as **specifications** of allowed behaviour

For example, a transition diagram (Mealy machine) or a state-transition table.

Abstraction function : checking correctness with respect to the written program (Finite State Machine accurately represents program behaviour $\equiv$ program correctly implements FSM abstraction).

Abstract (function) : takes a program state returns a FSM state

Example :

State variables: atCR, pos

Abstraction:

Abstract state	Concrete state		
	Lines	atCR	pos
e (Empty buffer)	3 – 13	0	0
w (Within line)	13	0	> 0
l (Looking for LF)	13	1	0
d (Done)	36	–	–

FSM:

	LF	CR	EOF	other
e	e / emit	l / emit	d / –	w / append
w	e / emit	l / emit	d / emit	w / append
l	e / –	l / emit	d / –	w / append

4 Data models

4.1 Define data dependence based on def-use pairs. Describe data dependence and control dependence graphs. Explain the general principle of dataflow analyses based on worklist algorithms, and the particular case of computing reaching definitions. Discuss the effect of pointers and procedures.

4.1.1 Define data dependence based on def-use pairs.

1. **def :**

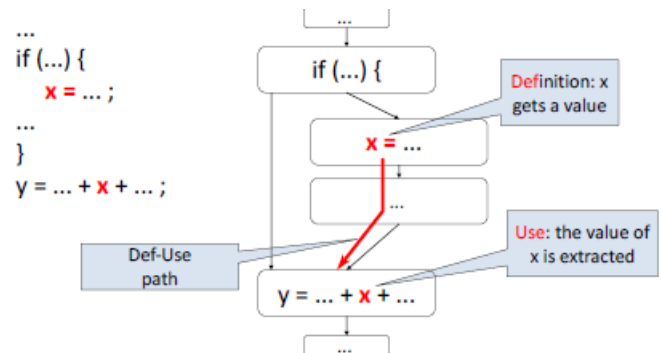
Point where a variable gets a value (Variable declaration, Variable initialization, Assignment, Value received by parameter)

2. **use :**

Point where a value from a variable is used (Expression, Conditional Statement, Parameter passing, returns)

⇒ **def-use pair** : pair of points

- from where a value is produced
- to where that value is used



⇒ **Data dependence based on def-use pairs** : Where does this value of x come from? What would be affected by changing this? ...

4.1.2 Describe data dependence and control dependence graphs.

Data dependence graph :

1. **Node :**

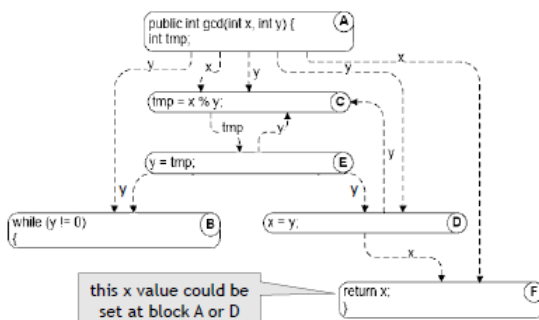
Program regions as in the control flow graph

2. **Edges :**

Def-use pairs labelled with the variable name

⇒ **Data dependence** :

P2 depends on P1 iff data values used in P2 can be defined in P1 (P1 is a **definition point**, P2 is an **use point**)



Control dependence graph :

1. **Node :**

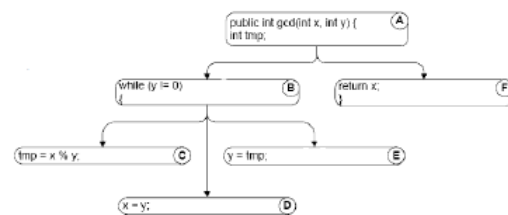
Program regions as in the control flow graph

2. **Edges :**

from **entry/branching** points to controlled blocks

⇒ **Control dependence** :

P2 depends on P1 iff P1 controls whether P2 executes (P1 is an **entry/branching point**, P2 is **any point**)

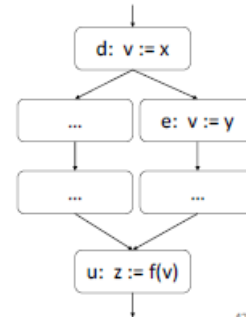


The difference with control flow is that blocks of CFG only follow other blocks but do not depends on them. Either of these blocks could be executed in either order.

4.1.3 Explain the general principle of dataflow analyses based on worklist algorithms, and the particular case of computing reaching definitions.

Let :

1. v_d, v_e definitions of variables v at points d, e
 2. u a point where v is used
- $\Rightarrow v_d$ **reaches** u (v_d is **reaching definition** at u)
iff
- (a) There is at least one control flow path from d to u
 - (b) There is no intervening definition of v on the path
- $\Rightarrow$ If a definition vd reaches p , and if v is not re-defined at p , then vd reaches n . **reaches** v_d iff it is on a control path from d
- $\Rightarrow (d, u)$ is a **def-use pair** of v iff v_d **reaches** u



Calculating Def-Use Pairs :

Even with loop-free paths, the number of paths in a graph can be **exponentially larger** than the number of nodes and edges. So we don't want to search every individual path but we want to summarize the reaching definitions at a node over all the paths reaching that node.

DF Algorithm :

- **Goal** : compute the reaching definitions at node n
- Suppose that node p is an immediate predecessor of node n
 - If p can assign variable v , $\rightarrow v_p$ reaches n .
We say the definition v_p is generated at p .
 - If a definition v_d reaches p , and if v is not redefined at p , $\rightarrow v_d$ reaches n .
- $Reach(n)$ = the set of definitions that reach n
- $ReachOut(n)$ = the set of definitions that exit n

Recursive equations for all nodes n
Fixed point computation

$$Reach(n) = \bigcup_{m \in pred(n)} ReachOut(m)$$

$$ReachOut(n) = (Reach(n) \setminus kill(n)) \cup gen(n)$$

$$gen(n) = \{ v_n \mid v \text{ is defined or modified at } n \}$$

$$kill(n) = \{ v_x \mid v \text{ is defined or modified at } n, x \neq n \}$$

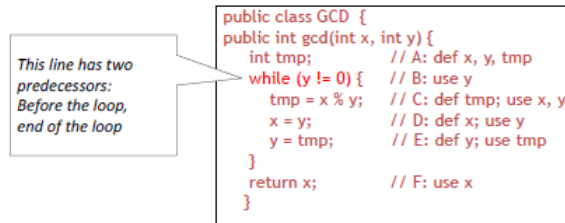
Worklist Algorithm :

```

foreach  $n \in nodes$  {
     $ReachOut(n) = \{\}$ 
}
worklist = nodes
while worklist  $\neq \{\}$  {
     $n = choose(worklist)$ 
     $worklist = worklist \setminus \{n\}$ 
     $oldVal = ReachOut(n)$ 
     $Reach(n) = \bigcup_{m \in pred(n)} ReachOut(m)$ 
     $ReachOut(n) = (Reach(n) \setminus kill(n)) \cup gen(n)$ 
    if  $ReachOut(n) \neq oldVal$  {
         $worklist = worklist \cup succ(n)$ 
    }
}

```

Example :



$$\text{Reach}(B) = \text{ReachOut}(A) \cup \text{ReachOut}(E)$$

$$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$$

$$\text{ReachOut}(E) = (\text{Reach}(E) \setminus \{y_A\}) \cup \{y_E\}$$

Particular cases :

1. **Avail expressions** : expression exp is **available** at node n iff for all paths to n, exp has been computed and not subsequently modified (used in compiler construction)

$$\text{Avail}(n) = \bigcap_{m \in \text{pred}(n)} \text{AvailOut}(m)$$

$$\text{AvailOut}(n) = (\text{Avail}(n) \setminus \text{kill}(n)) \cup \text{gen}(n)$$

$$\text{gen}(n) = \{ \text{exp} \mid \text{exp is computed at } n \}$$

$$\text{kill}(n) = \{ \text{exp} \mid \text{exp has variables assigned at } n \}$$

2. **Live variables** : A variable v is **live** at node n iff on some execution path from n, v is used before it is changed

$$\text{Live}(n) = \bigcup_{m \in \text{succ}(n)} \text{LiveOut}(m)$$

$$\text{LiveOut}(n) = (\text{Live}(n) \setminus \text{kill}(n)) \cup \text{gen}(n)$$

$$\text{gen}(n) = \{ v \mid v \text{ is used at } n \}$$

$$\text{kill}(n) = \{ v \mid v \text{ is modified at } n \}$$

Classification of analyses :

- **Forward/backward** : a node's set depends on that of its predecessors/successors
- **Any-path/all-path** : a node's set contains a value iff it is coming from any/all of its inputs

	Any-path ($\cup$)	All-paths ($\cap$)
Forward (pred)	Reach	Avail
Backward (succ)	Live	"inevitable"

4.1.4 Discuss the effect of pointers and procedures.

1. **Arrays and pointers** : introduce **uncertainty** : do different expressions access the same storage ?
For example : $a[i]$ is the same as $a[k]$ when $i=k$ or $a[i]$ same as $b[i]$ when $a=b$.

Solution :

- (a) **Any-path** : gen sets contains all potential aliases, kill sets contain only what is definitely modified.
- (b) **All-path** : vice versa

2. **Procedures : interprocedural** (Across several methods or procedures) data flow analysis has cost/-precision trade-offs critical and difficult : context sensitivity and flow sensitivity.
For example : Reach, Avail,... are flow-sensitive and **intraprocedural** (Within a single method or procedure) analyses which cost $O(n^3)$ for one procedure (reasonably cheap) but what about doing flow-sensitive interprocedural analyses? $\rightarrow O(n^3)$ on the whole program : prohibitive! So many interprocedural flow analyses are flow-insensitive (it is often good enough, e.g. type checking)

5 Functional testing

5.1 Define test case, test obligation, adequacy criterion, test satisfaction. Define functional and structural testing. Explain category-partition testing. Explain pair-wise and n-wise testing. Explain catalog-based testing.

5.1.1 Define test case, test obligation, adequacy criterion, test satisfaction.

- | | |
|---|---|
| <p>1. Test case :
a set of inputs, execution conditions, and a pass/fail criterion (criterion = functional vs structural)</p> <p>2. Test obligation :
a partial test case specification (a requirement to be satisfied by one or more test cases.), requiring some property deemed(=considered) important to thorough testing</p> | <p>3. Adequacy criterion :
a predicate that is true (satisfied) or false (not satisfied) of a <program, test suite> pair.
= Set of test obligations</p> <p>4. Test satisfaction :
A test suite (set of test cases) satisfies an adequacy criterion if :
 <ul style="list-style-type: none"> • all the tests succeed (pass) • every obligation is satisfied by at least one test case </p> |
|---|---|

Infeasible Criterion : Sometimes no test suite can satisfy a criterion for a given program

⇒ Solution : eliminate infeasible test obligations = **Undecidable in the general case !**

⇒ **Solution :** measure fraction of obligations covered = **Coverage**

5.1.2 Define functional and structural testing.

- | | |
|---|--|
| <p>1. Functional testing : (black box, closed box)</p> <ul style="list-style-type: none"> • program content is unknown or ignored → tests according to the specifications derive from model (FSM, grammar, CFG/DFG) • Test input/output behavior • Obligations from functional specifications <p>= systematic testing (Select inputs that are especially valuable, Different classes, limit cases, special values)</p> | <p>2. Structural testing : (white box, clear box)</p> <ul style="list-style-type: none"> • Program content is visible and observed → tests according all the program statement (100% coverage if all possible) • Test internal operation • Obligations from program code |
|---|--|

5.1.3 Explain category-partition testing.

Category-partition testing : 3 steps

1. **Decompose the specification into units, parameters, categories**
 - (a) Identify independently testable unit
 - (b) For each unit, identify parameters and environment elements
 - (c) For each parameter, identify categories (characteristics) → Not a trivial task! No hard-and-fast rules, reflect test designer's judgment,...
2. **Identify relevant choices (values) for each category**
 - (a) Identify classes of values for each category (ignore interactions between different categories)
 - (b) Boundary value testing

- i. extreme values within a class
- ii. values just outside the class
- iii. interior (non-extreme) values
- (c) Erroneous condition testing : values outside the normal domain of the program

3. **Introduce constraints** : combination of values for each category corresponds to a test case specification. Large : product of category sizes, most of which are impossible! Introduce constraints to rule out impossible combinations, reduce the size of the test suite if too large

5.1.4 Explain pair-wise and n-wise testing.

Category-partition testing is a systematic approach to generate combinations, but the test suite size grows very rapidly with number of categories, even with constraints! The idea of pairwise testing is to use a non-exhaustive approach

1. Pair-wise testing : all pairs of choices

Pairwise combination : generate combinations that efficiently cover all pairs (triples,... in case of n-wise) of choices. Justified by the fact that most failures are triggered by a single value or a combination of a few values, so covering pairs (triples,...) reduces the number of test cases, but reveals most faults.

Complexity : For N categories with M choices each :

- **All combinations** = $O(M^N)$ test cases $\rightarrow$ exponential in number of categories
- **All pairs** = $O(M^2 \log(N))$ test cases $\rightarrow$ logarithmic in number of categories

2. N-wise testing : same as pairwise but all N-tuples of choices

5.1.5 Explain catalog-based testing.

- Deriving value classes requires human judgment, catalog-based testing aims to gather experience in a systematic collection.
- Catalogs list important cases for each possible type of variable

• Benefits :

1. speed up the test design process
2. routinize many decisions, better focusing human effort
training and reduce human error

• Process :

1. analyze the initial specification to identify simple elements : pre-conditions, post-conditions, definitions, variables, operations
2. derive a first set of test case specifications from pre-conditions, post-conditions and definitions
3. complete the set of test case specifications using test catalogs

Catalog :

1. Each entry = a kind of element that can occur in a specification
2. Each entry is associated with a list of generic test case specifications

Example :

catalog entry Boolean two test case specifications : true, false Label in/out indicate if applicable only to input, output, both

Boolean	
True	in/out
False	in/out
Enumeration	
Each enumerated value	in/out
Some value outside the enumerated set	in
Range L ... U	
L-1	in
L	in/out
A value between L and U	in/out
U	in/out
U+1	in
Numeric Constant C	
C	in/out
C-1	in
C+1	in
Any other constant compatible with C	in

6 Structural testing

6.1 Define statement, branch and condition coverage, compound conditions and MC/DC, and compare their strengths. Define path coverage, discuss limitations and show some practical path coverage criteria. Define data flow coverage criteria. Discuss the problems of aliases and infeasibility.

6.1.1 Define statement, branch and condition coverage, compound conditions and MC/DC, and compare their strengths.

1. Statement coverage :

each statement must be executed at least once.
Motivation : a fault in a statement can only be revealed by executing the faulty statement

$$C_{stmt} = \frac{\#executed\ statement}{\#statement}$$

NB : CFG nodes $\neq$ statements : may represent basic blocks, multiple statements or parts of statements. Difference in granularity, not in concept. 100% node coverage = 100% statement coverage

2. Branch coverage :

each branch must be executed at least once. Variant : edge coverage $\rightarrow$ each edge in the CFG. In a graph : traversing all edges $\Rightarrow$ visiting all nodes.

$$C_{stmt} = \frac{\#executed\ statement}{\#statement}$$

NB : 100% branch coverage $\Rightarrow$ 100% statement coverage (but not the converse!)

Branch coverage = **decision coverage** : each decision must be true and false at least once (decision = top-level boolean expression)

3. Condition coverage :

considers case analysis in more detail : individual conditions in a boolean decision are tested. Each **basic condition** must be true and false at least once.

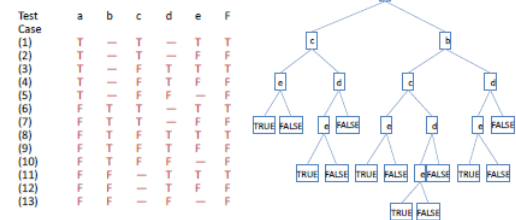
$$C_{bcond} = \frac{\#executed\ true\ basic\ conds + \#executed\ false\ basic\ conds}{2 * \#basic_conds}$$

Note : basic condition coverage can be satisfied without satisfying branch coverage. Branch and basic condition are not comparable $\rightarrow$ neither implies the other

4. Compound conditions coverage :

each possible evaluation of each decision must be taken at least once : all branches of the decision tree. Note : exponential complexity. N conditions $\Rightarrow O(2^N)$ test cases

$F = (((a || b) \&\& c) || d) \&\& e$



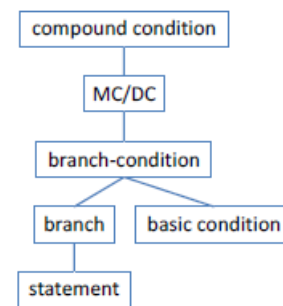
5. MC/DC (Modified condition/decision) coverage :

each basic condition is shown to independently affect the decision

Motivation : test important combinations of conditions, without exponential blowup

Requires, for each basic condition C in a decision D, two test cases such that :

- Values of all evaluated conditions except C are the same and
- D evaluates to true for one and false for the other



N basic conditions $\Rightarrow$ N+1 test cases (linear complexity)

$F = (((a \parallel b) \&\& c) \parallel d) \&\& e$

Test Case	a	b	c	d	e	F
(1)	<u>T</u>	--	<u>T</u>	--	<u>T</u>	T
(2)	<u>T</u>	--	<u>T</u>	--	<u>F</u>	F
(3)	T	--	F	<u>T</u>	T	T
(5)	T	--	<u>F</u>	<u>F</u>	--	F
(6)	F	<u>T</u>	<u>T</u>	--	T	T
(13)	<u>F</u>	<u>F</u>	--	F	--	F

(5.) A good balance of thoroughness and test size (and therefore widely used)

Is basic condition coverage, decision (=branch) coverage, plus one additional condition (M) : every condition must independently affect the decision's output

6.1.2 Define path coverage, discuss limitations and show some practical path coverage criteria.

- **Path coverage** : decision and condition coverage : consider individual program decisions. Many more paths than branches. Each path must be executed at least once

$$C_{path} = \frac{\#executed\ paths}{\#paths}$$

- **Limitations** :

1. Program with loops $\Rightarrow$ infinite number of paths! Full path coverage is therefore usually impossible to satisfy.
2. Feasible criterion : partition infinite set of paths into a finite number of classes, by limiting :
 - (a) the number of traversals of loops
 - (b) the length of the paths to be traversed
 - (c) the dependencies among selected paths

- **Some practical path coverage criteria** :

- **Boundary interior path coverage** : each path up to the first repeated node must be executed at least once
 1. Group together paths that differ only in the subpath they follow when repeating the body of a loop
 2. construction : unfold the CFG up to the first repeated node
 3. limitations : the number of paths can still grow exponentially
- **Cyclomatic coverage** : a basis of independent paths must be executed
- **Loop boundary coverage** : each loop body must be iterated zero times, one time and more than one time at least once.

6.1.3 Define data flow coverage criteria.

1. **Motivation for data flow testing** (not really asked) : statement and branch coverage don't test interactions, path-based coverage require impractical number of test cases and only a few paths uncover additional faults. $\rightarrow$ We need to distinguish important paths. Intuition : statements interact through data flow (value computed in one statement, used another. Bad value computation revealed only when it is used)
2. **All DU (def-use) pairs** : each DU pair is exercised at least once
3. **All DU paths** : each simple (non looping) DU path is exercised at least once
4. **All definitions** : for each definition, some DU pair is exercised at least once (every computed value is used somewhere)

6.1.4 Discuss the problems of aliases and infeasibility.

- **Problem with aliases** : which references are (always or sometimes) the same ?
Ex : $p=x; \dots; *p=99 \rightarrow *p$ is an alias of x
- **Problem of infeasibility** : with conditional statement, it is possible that a variable is defined and never used = infeasibility problem. This problem is relevant, combinations of elements matter, it is impossible to decide feasible paths. In practice, we achieve reasonable coverage, but full coverage is usually unattainable. Number of paths is exponential in worst case, but often linear. But testing all DU paths is more often impractical since attainability is an undecidable problem

7 Model-based testing

7.1 Explain the principles of model-based testing. Explain path-insensitive and path-sensitive state machine coverage criteria. Discuss coverage criteria for decision structures and grammars.

7.1.1 Explain the principles of model-based testing.

1. It is used in order to test structured specifications (State machines, tables, graphs, grammars,...)
2. Consist on devising test cases to check actual behavior against behavior specified by the model
3. Coverage similar to structural testing, but applied to specification and design model

7.1.2 Explain path-insensitive and path-sensitive state machine coverage criteria

1. Covering finite state machine :

Finite state machines : for describing behavior that depends on sequences of events or stimuli

- **State coverage** : every state in the model should be visited at least once
- **Transition coverage** : every transition in the model should be traversed at least once (this is the most commonly used criterion)

Transition coverage assumes that **transitions depend only on current state** and not on path to reach the state. This is not always true → needs path-sensitive criteria

2. Path-sensitive criteria :

- **Single state path coverage** : traverse each subpath that reached states at most once
- **Single transition path coverage** : traverse each subpath that reaches transitions at most once
- **Boundary interior loop coverage** : traverse each distinct loop the minimum, an intermediate, and the maximum or a large number of times (the most common)

7.1.3 Discuss coverage criteria for decision structures and grammars.

• Coverage criteria for decision structures :

A representation of a function $result = F(conditions)$

n conditions → 2^n possible combinations (decision tables/trees, flow charts).

Treat as **Boolean expressions**.

Covering : Apply condition/decision-based criteria :

1. **Basic condition coverage** : a test case for each column
2. **Compound condition coverage** : a test case for each (possible) combination of basic conditions
3. **Modified coverage (MC/DC)** : add columns that differ in one input row and in outcome, merge compatible columns, a test case specification for each column

• Coverage criteria for decision grammars :

Useful for sequences and nested structure. Test cases are string generated from the grammar :

1. **Production coverage** : each production must be used at least once
2. **Boundary condition coverage** : each recursive production must be used (min, min+1, max-1, max) times, where min and max are set for each production (similar to boundary interior path)

Test cases generated depend on generation strategy :

1. productions with non-terminals first → few, large test cases
2. productions with terminals first → many, small test cases

8 Object-oriented testing

8.1 Explain the principles of testing of object-oriented software, at the unit and integration level. Discuss structural coverage criteria for intra- and inter-class testing. Discuss the issues of test oracles, polymorphism and exception handling.

8.1.1 Explain the principles of testing of object-oriented software, at the unit and integration level.

For an object oriented software : **unit** = single class (or cluster of strongly related classes e.g. exceptions)

⇒ **unit testing** = intra-class testing

⇒ **integration testing** = inter-class testing

8.1.2 Discuss structural coverage criteria for intra- and inter-class testing.

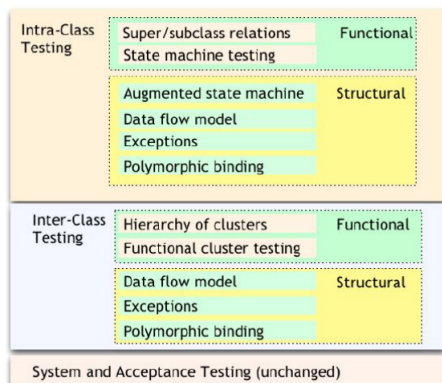
- **Inter-class testing** (state machine) :
 - Test interactions between classes
 - Bottom-up integration, according to “depends” relation
 - Use/include hierarchy :
 1. A uses B = A makes method calls on B
 2. A includes B = A objects include references to B objects, ignore inheritance, abstract classes
 - Bottom-up integration, clustering

- **Intra-class testing :**

Basic idea :

- Objects have a state
- Methods calls are state transitions
- Test cases are sequences of method calls

State machine can be derived from specification and code → Model-based testing, state/transition coverage



- **Inter-class structural testing :**

DU pair structural testing :

- working bottom-up in dependence hierarchy (leaf class, then classes that use leaf classes, ...)
- classify each method :
 1. **inspectors** : use, but do not modify, object state
 2. **modifiers** : modify, but not use, object state
 3. **inspector/modifiers** : use and modify object state
 4. treating a whole object as variable (not each field)
- Treat inspector calls as uses, modifier calls as defs

- **Intra-class structural testing :**

- As for procedural software, start with functional testing (from specifications), then complete with structural testing (from the code)
- use control flow graph : each method + node for class + edges → method, method → class ⇒ control flow through sequences of method calls

8.1.3 Discuss the issues of test oracles, polymorphism and exception handling.**1. Test oracles** must be able to check the correctness of a test execution

- **Correct output** : OK, can be checked
- **Correct new state** : not accessible, encapsulation

Accessing the state :

- **Intrusive approach** : use language constructs, add inspector methods → breaks encapsulation, may produce undesired results
- **Equivalent scenarios approach** : generate equivalent sequences of method calls, compare the final states of the objects

2. Polymorphisme : Combinatorial explosion problem (dynamic bindings). When testing a child class, we would like to test only what is needed not what has been tested in the parent class (any method whose behavior has been changed)

Testing history approach :

- (a) Track test suites and test executions
 - determine which new tests are needed
 - determine which old tests must be re-executed
- (b) New and changed behavior
 - new methods must be tested
 - redefined methods must be tested, we can partially reuse test suites
 - unchanged methods need not be retested
- (c) Executing test cases is usually cheap, it may be simpler to re-execute the full parent test suite

3. Exceptions :**(a) Exceptions**

- implicit control flows
- may be handled by different handler

Impractical to treat exceptions like normal flow

- Too many flows : every exception source times every exception handler

(b) Program error exceptions : test to prevent them, not to handle them.**(c) Explicit throws** : test with respect to every handler on call stack**(d) Local exception handlers** : test the exception handler**(e) Non-local exception handlers :**

- difficult to determine all <source, handler> pairs
- design rule : if a method propagates an exception, the method call should have no other effect
- test all sources, all handlers (but not all pairs)

9 Fault-based testing

9.1 Explain the principles and assumptions of mutation testing. Give examples of mutation operators. Discuss fault-based coverage measures. Discuss ways to reduce the cost of mutation testing. explain fault estimation using seeded faults or independent test groups.

9.1.1 Explain the principles and assumptions of mutation testing.

• **Principles :**

1. A mutation is a syntactic change (a seeded fault) → ex : change($i < 0$) to ($i \leq 0$)
2. A mutant is a copy of a program with a mutation (valid mutant = syntactically correct)
3. Run test suite on all the mutants
4. A mutant is killed if it fails on at least one test case
5. If many mutants are killed then the test suite is effective at finding real faults

• **Assumptions :**

1. **Competent programmer hypothesis :**
Programs are assumed to be nearly correct. Real faults are small variations from correct program. Mutants are reasonable models of real faults
2. **Coupling effect hypothesis :**
Tests that find simple faults also find more complex faults. Even if mutants are only simple faults, a test suite that kills mutants is good at finding complex faults too

9.1.2 Give examples of mutation operators.

1. **crp** : constant for constant replacement
2. **Competent programmer hypothesis** : programs are nearly correct
 - real faults are small variations from the correct program
 - mutants are reasonable models of real faults
3. **Coupling effect hypothesis** : Tests that find simple faults also find more complex faults even if mutants are simple faults, a test suite that kills mutants is good at finding complex faults
too($x < 5$) → ($x < 12$)
4. **rpr** : relational operator replacement
($x \leq 5$) → ($x < 5$)
5. **vie** : variable initialization elimination
int x = 5; → int x;

9.1.3 Discuss fault-based coverage measures.

There are two possible reasons that a mutant survive :

1. **The mutant is equivalent to the original program.** The mutation does not change the behavior. The seeded fault is not really a fault
2. **The test suite is inadequate.** The mutant could have been killed, but was not. Adding a test case for just this mutant is a bad idea! We care about the real bugs, not the fakes!

Fault-based coverage : All non-equivalent mutants are killed by at least one test case

$$C_{Fault} = \frac{\#killed\ mutant}{\#non - equiv\ mutant}$$

9.1.4 Discuss ways to reduce the cost of mutation testing.

Equivalent mutants are hard to determine (undecidable in the worst case) and there are lots of mutants

- ⇒ high cost
- ⇒ grows with the square of program size
- ⇒ running each test case on each mutant is expensive

Solutions :

1. **Weak mutation** : observe states of program and mutant, kill as soon as a difference is found (do not wait for test completion)
2. **Meta-mutant** : mutant with several seeded faults, with mechanism to activate the mutants (check several mutants in one test run)
3. **Statistical mutation** : create a random sample of mutants (OK for assessing a test suite)

9.1.5 Explain fault estimation using seeded faults or independent test groups.**1. Seeded faults :**

How many remaining (natural) faults N ?
Intentionally seed S faults in the program
Run the tests

- s discovered seeded faults
- n discovered natural faults

Hypothesis : same effectiveness $n/N = s/S$

$$\Rightarrow N = S.n/s$$

2. Independent test groups :

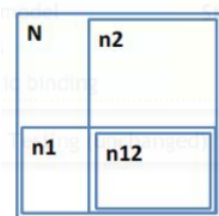
If we don't know typical faults?
Split tests in two groups $E1$, $E2$

- $n1$ faults detected by $E1$
- $n2$ faults detected by $E2$
- $n12$ faults detected by both $E1$ and $E2$
- N faults in total

Hypothesis : effectiveness of $E1$ is the same on all faults as on faults detected by $E2$:

$$\Rightarrow n1/N = n12/n2$$

$$\Rightarrow N = n1.n2/n12$$



10 Test execution

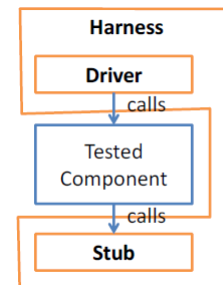
10.1 Describe the principles of scaffolding for test execution. Discuss different types of test oracles. Describe the nature and objectives of unit, integration, system, acceptance and regression testing activities. Discuss different integration testing strategies. Explain regression test selection and prioritization.

10.1.1 Describe the principles of scaffolding for test execution.

Scaffolding (échafaud) : code produced to support development activities especially testing. Not part of the product May be temporary

It include :

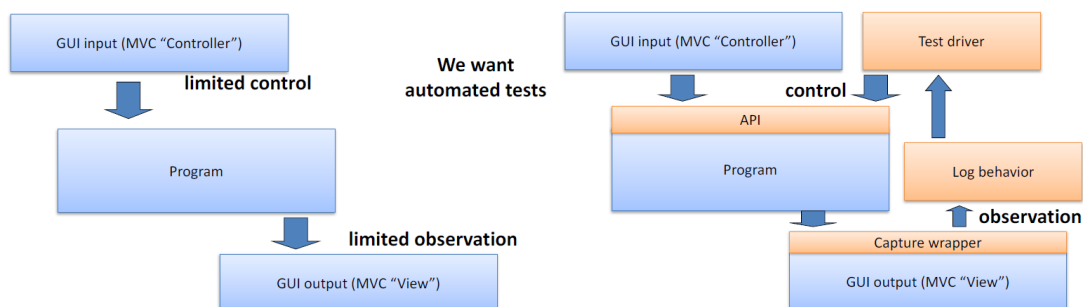
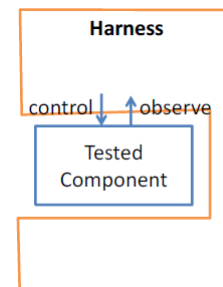
1. **Test harness** : environment in which the component is tested. Ex : Software simulation of a hardware device
2. **Test driver** : calls the component. Applies the test cases A “main” program for running a test
3. **Test stubs** : called by the component. Substitutes for called components

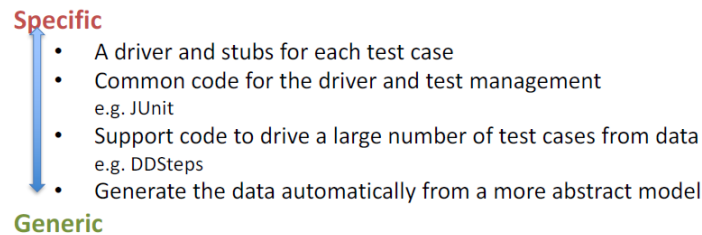


The scaffolding must provide :

1. **Controllability** : allow to execute test cases
2. **Observability** : allow to judge the outcome of tests

⇒ May require additional interfaces, drivers





10.1.2 Discuss different types of test oracles.

Did this test case succeed, or fail? **Oracle** : software that determines whether a test passed or failed.

Better than manual checking

1. More efficient
2. More reliable
3. More capable (e.g timing, large data)

Oracles should ideally :

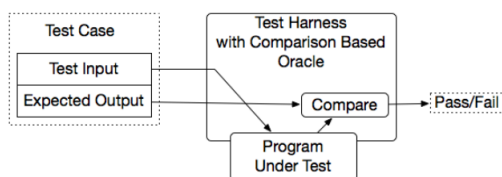
- report **PASS** for **all correct** executions and
- report **FAIL** for **all incorrect** executions

Partial oracles :

- report PASS for all correct executions and
- may report PASS for some incorrect executions

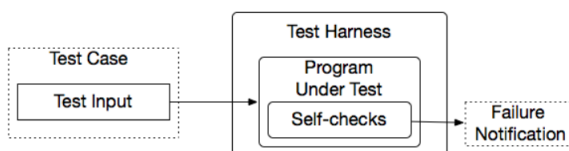
No false alarms (FAIL on correct executions). Several partial oracles may be more effective than one complete oracle.

Comparison-Based Oracle :



Oracle compares actual to expected output, reports if (actual = expected) then PASS else FAIL. Fine for a small number of hand-generated test cases. E.g. JUnit test cases : assertEquals(actual, expected)

Self-Checks as Oracle :



Oracle as self-checks in the program (assertions) judge correctness without predicting results

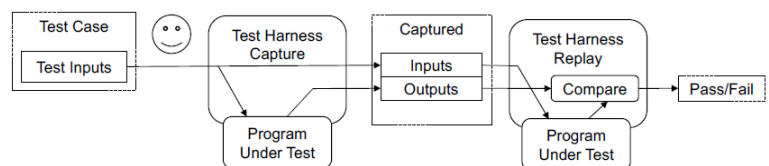
- + Usable with large, automatically generated test suites
- Often only a partial check.
e.g., structural invariants of data structures

Assertions as Oracle :

- **Invariants on data structures** e.g. `assert 0 <= size & size <= a.length`
- **Pre- and post-conditions**
e.g. `assert k != null; v = dict.get(k); assert dict.contains(k, v);`

May need to deal with quantifiers. implement as iteration $\Rightarrow$ does not scale well or sample some elements (partition testing!)

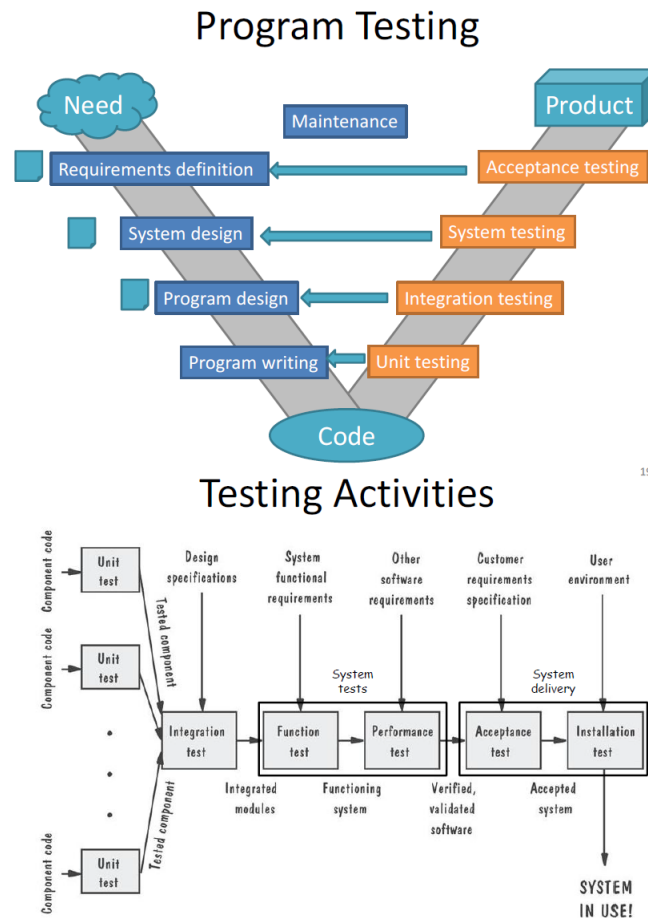
Capture and Replay :



- **Capture** a manually run test case, sequence of inputs, outputs
- **Replay** it automatically with a comparison-based oracle and compare actual to captured outputs

Reusable only until a program change invalidates it
lifetime depends on abstraction level of input and output

10.1.3 Describe the nature and objectives of unit, integration, system, acceptance and regression testing activities.



- **Unit Testing** : Aka module testing, component testing
 - Testing : feed inputs, check valid outputs
 - Code reviews, analyses : check internal data structures, logic
- **Integration Testing** :
 - Assemble components together
 - Check correct interaction
- **Function Testing**
 - The whole system
 - Check that the behaviour conforms to functional requirement specifications
- **Performance Testing**
 - The whole system
 - Check that the behaviour conforms to non-functional requirement specifications
- **Acceptance Testing**
 - The whole system
 - Test using the system with the customer
 - Check that the behaviour conforms to the customer requirement documentation.
- **Regression Testing**
 - Regression = loss of correct functionality after a change (Adding new features ; changing, adapting conditions ; bugs fixing)
 - **Regression testing** = re-executing tests after any change to detect regressions
 - Can be a major cost of software maintenance Sometimes much more than making the change

10.1.4 Discuss different integration testing strategies.

Depends on **system characteristics AND customer expectations**

	Bottom-up	Top-down	Modified top-down	Big-bang	Sandwich	Modified sandwich
Integration	Early	Early	Early	Late	Early	Early
Time to basic working program	Late	Early	Early	Late	Early	Early
Component drivers needed	Yes	No	Yes	Yes	Yes	Yes
Stubs needed	No	Yes	Yes	Yes	Yes	Yes
Work parallelism at beginning	Medium	Low	Medium	High	Medium	High
Ability to test particular paths	Easy	Hard	Easy	Easy	Medium	Easy
Ability to plan and control sequence	Easy	Hard	Hard	Easy	Hard	hard

10.1.5 Explain regression test selection and prioritization.

Problems of Regression Test : Maintaining the test suite (Obselete vs Redudant tests)

⇒ Cost of re-testing = often proportional to the product size

⇒ **Select** or **prioritize** test cases

Test case selection : do not execute some test cases When test cases are expensive to execute (special equipment, or long run-times, or manual intervention)

- **Principle** : execute only test cases related to elements that were affected by the change

⇒ **Code-based selection** : only execute test cases that execute changed or new code

1. **Independent** : a test case can't find a fault in code it doesn't execute

Variants : changed CFG nodes (control-flow) changed def-use pairs (data-flow)

2. Needs to record elements touched by each test case and modified by each change

⇒ **Specification-based selection** : only execute test cases that test new and changed functionality

Not independent : a test case that is not "for" a changed feature X might find a bug in feature X

⇒ prefer prioritization rather than selection

Test case prioritization : execute some test cases less often When a very large test suite cannot be executed every day

- **Basic idea** :

Execute all test cases, eventually
Execute some sooner than others

- Possible priority schemes :

1. **Specification-based** : priority to test cases related to changed and added features
2. **Round robin** : Priority to least-recently-run test cases
3. **Track record** : Priority to test cases that have detected faults before. They probably execute code with a high fault density
4. **Structural** : Priority for executing elements that have not been recently executed
Can be coarse-grained : features, methods, files, ...

11 Program analysis = Execute tests

11.1 Describe principles of program inspection. Define static and dynamic program analysis. Describe the principles of symbolic program execution. Describe the principles of program verification using symbolic execution. Discuss contract-based reasoning on procedures and data structures. Describe the principles of symbolic testing and dynamic program analyses. Discuss lockset analysis.

11.1.1 Describe principles of program inspection.

Systematic, detailed review of artifacts (= code but also specifications, documentation, tests, ...) to find defects and assess quality.

Used for :

1. Find and remove defects
2. Incentive to produce good code
3. Share coding norms and practices
4. Familiarize new staff with the code

Team : the programmer(s) + experts (+ usage of **checklist**)

with different perspectives : junior and senior engineers, test, managers, analysts, architects, ...

Moderator : external senior manager

11.1.2 Define static and dynamic program analysis.

1. **Static analysis** : examine source code. Examine the complete execution space but may lead to false alarm
2. **Dynamic analysis** : examine execution traces. No infeasible path problem but cannot examine the execution space exhaustively

11.1.3 Describe the principles of symbolic program execution.

Execute the program with symbolic values. Variables receive symbolic values. Execution paths accumulate symbolic conditions → Bridges program behavior to logic (program behavior becomes logic)

1. Values are symbolic expressions
2. Executing statements computes new expressions
3. For branching statements, both branches are possible (non-determinism). Need to record the condition for the execution of each branch
4. Path accumulate conditions which may become extremely complex. We can simplify it by replacing a complex condition P with a weaker condition W such that $P \Rightarrow W$. W describes the path with less precision (W is a summary of P)
5. To reason about program behavior in a loop, we can place an invariant. Each time program execution reaches the invariant W , we can weaken the execution condition P to W :
 - (a) check that $P \Rightarrow W$
 - (b) substitute W for P
6. If :
 - (a) Every loop contains an invariant

- (b) there is an assertion at the beginning of the program
- (c) there is an assertion at the end of the program

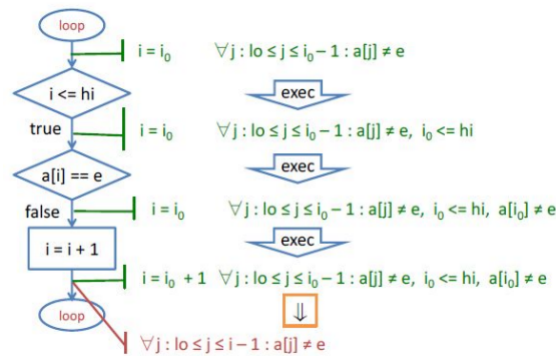
Then, every possible execution path is a sequence of segments (basic paths) from one assertion to the next.

- **Precondition** : assertion at the beginning of a segment
- **Postcondition** : assertion at the end of the segment

11.1.4 Describe the principles of program verification using symbolic execution.

To verify program correctness, we do the following :

1. Verify for each basic path :
 - (a) Starting from the precondition
 - (b) Executing the program segment
 - (c) the postcondition holds at the end
2. Then the execution of any path is correct



11.1.5 Discuss contract-based reasoning on procedures and data structures.

- **On procedure** : Compositional reasoning

Follow the hierarchical structure of a program :

- at a small scale (within a single procedure)
- at a larger scales (across multiple procedure ...)

Hoare triple : [pre] block [post]

- if pre is satisfied at the entry to the block, then the post should be satisfied after execution of the block
- ⇒ summarize the effect of a block of program by a contract [pre] block [post]
- ⇒ Prove that block satisfies pre/post then use the contract wherever the block is used

- **On data structures** :

Data structure module = data (encapsulated) + operations = variables + procedures (methods)
Contract =

1. **Abstraction function** "abs" : relates data structures D to an abstract model abs(D)
e.g. abs : Dictionary → <key, value>
2. **Structural invariant** "ok" : data structure characteristics that must be maintained e.g. ok : Dictionary → bool

Contract for Dictionary.get :

```

[<k,v> in abs(dict) and
  ok(dict)]
o = dict.get(k)
[o = v and ok(dict)]
  
```

11.1.6 Describe the principles of symbolic testing and dynamic program analyses.• **Symbolic testing :**

- abstract variables with few symbolic values
- apply symbolic execution
- no need to follow all paths
 - explore paths to a limited depth
 - prune exploration by some criterion
- sensitivity :
 - Symbolic testing is path sensitive : different symbolic states from paths to the same location
 - Symbolic testing is partly context sensitive : different symbolic states implies different call sequences
 - This is a strength of symbolic checking
 1. detailed description of how fault is reached
 2. costly
 3. reduce costs by memoizing entry and exit conditions

• **Dynamic program analysis :**

- track program execution
 - instrument program to trace memory access the state of each memory location
 - detect accesses incompatible with the current state
 - access unallocated memory
 - read from uninitialized memory
 - array bounds violations
- amplify sensitivity of testing to detect to detect potential data races (two threads access a location, and at least one is a write, and there is no lock protecting that location).

11.1.7 Discuss lockset analysis.

- **Lockset discipline :** Every shared variable must be protected by a lock
- **Dynamic lockset analysis :** detects violation of the locking discipline
- **Algorithm :**

Identify set of locks held by threads when accessing each shared variable
 For each variable x , a set $\text{Lockset}(x)$
 INIT: $\text{Lockset}(x) = \{\text{all locks}\}$
 thread A accesses x : $\text{Lockset}(x) = \text{Lockset}(x) \cap \text{Locks}(A)$
 END: if $\text{Lockset}(x) = \{\}$ report ERROR (no lock consistently protects x)

Thread	Program trace	Locks held	Lockset(x)	
thread A	lock(lck1)	{}	{lck1, lck2}	INIT: all locks for x
	$x = x + 1$	{lck1}		lck1 held
	unlock(lck1)		{lck1}	Intersect with locks held
thread B	lock(lck2)	{}		
	$x = x + 1$	{lck2}		lck2 held
	unlock(lck2)		{}	Empty intersection potential race
		{}		

12 Finite state analysis

12.1 Describe the principles of finite-state analysis. Define safety and liveness properties and discuss their verification. Discuss the model size and correspondence problems. Discuss intensional representations. Discuss iterative model refinement. Discuss data model verification.

12.1.1 Describe the principles of finite-state analysis.

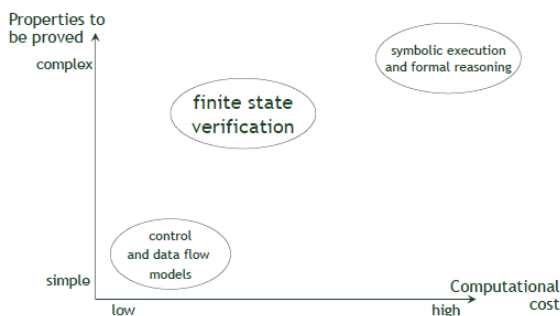
Finite state verification :

- Prove some significant properties on a finite model of the infinite execution space
- Techniques from symbolic execution and formal verification : Finite models of the potentially infinite state space
- Iterative process :
 1. Prepare a model and specification
 2. Repeat :
 - (a) Attempt verification
 - (b) Receive reports of impossible or unimportant faults
 - (c) Refine the specification and/or the model
 3. Until no impossible or unimportant faults
- is complementary to testing (can find bugs that are extremely hard to test for [concurrency, race conditions] but is limited in scope)

12.1.2 Define safety and liveness properties and discuss their verification.

Safety :

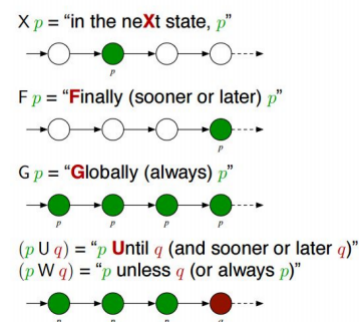
- Bad things should not happen
- Example :
1. invariant violation, assertion violation
 2. mutual exclusion : two process should not modify a variable at the same time
- P S [Q] : partial correctness
- Specify with `assert(...)`
 - **Verify with reachability (easy)**



Liveness :

- Good things should eventually happen
- Example :
1. response : if I push the button, eventually the elevator should arrive
 2. fairness : all enabled threads get executed
 3. program termination
- Specify in **temporal logic**

Temporal logic: LTL



- Verify with automata, repeated reachability (more expensive)

12.1.3 Discuss the model size and correspondence problems.

1. The **size of the state model** growth very quickly and a the size of the model can became too large very quickly.

With $\#K$ process with $\#N$ states each, the number of global state is : $\#K^{\#N}$

2. **Model correspondence problem** : Consistency between model and program ?

- (a) **Model extracted from the program**

⇒ verify the **extraction** procedures (once for all)

- Challenge : right level of detail
 - i. all details ⇒ state space explosion
 - ii. missing details ⇒ “false alarm” reports

- (b) **Program generated from the model**

⇒ verify the **generation** procedures (once for all)

Most applicable within well-understood domains

- (c) **Model written (partially or entirely) by hand**

⇒ check **conformance** by (model-based) testing

12.1.4 Discuss intensional representations.

Enumerating (and storing) all reachable states cost a lot, it limites factor of finite state verification. The alternative is to use intensional (symbolic) representations which describes sets of reachable states without enumerating each one individually.

- Intensional representations may be more compact than the set they represent

$$\{x \in N | x \bmod 2 = 0 \wedge 0 < x < 1000\} \rightarrow 500 \text{ elem}$$

This is only because of structure or regularity in the set that is captured by the representation !

- Unstructured, irregular sets will necessarily have a larger intensional representation
- **Information theory** : representing subsets of N elements (2^N possibilities) requires $O(N)$ in average

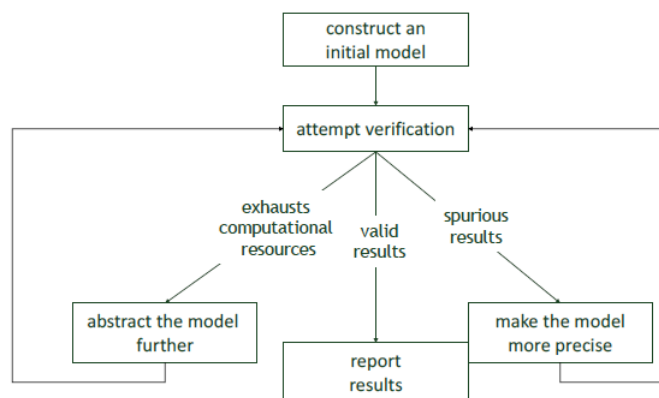
12.1.5 Discuss iterative model refinement.

Construction of finite state models (balancing precision and efficiency).

Often the first model is unsatisfactory

- report unfeasible failures
- exhaust resources before producing any result

⇒ Idea : improve the model and restart (Finite state verification as iterative process)



12.1.6 Discuss data model verification.

Many information systems are characterized by

1. simple program logic and algorithms
2. complex data structures

A key element is the data model (class and object diagrams + OCL assertions) = sets of data and relations among them.

Challenge : prove that

1. individual constraints are consistent
2. together they ensure the desired properties of the system as a whole

In a complex data models, we use the same general verification principles

1. systematic analysis of models
2. thorough testing is impractical

The difficulty is to consider all the possible combinations of choices in a complex data model. (take the website as an example)

13 Software measurement : size

13.1 State the general principles of software measurement. Discuss size measurement metrics based on lines of code. Explain functional size measurement using function points.

13.1.1 State the general principles of software measurement.

Principles :

- **Software measurement** : deriving a numeric value of a property of a software product or a process. (To allow comparison)
- **Measurement** (more general) : mapping from the empirical world to the formal world
- **Metrics** : a means of measurement of a property of a software product or a process.
- **Objective** : understand, control and improve the quality of a software.
- **Attributes** :
 1. **Internal** (structural) : size, complexity
 2. **External** (functional) : quality, reliability

13.1.2 Discuss size measurement metrics based on lines of code.

Note : A size measurement should be non-negative, zero iff empty and additive.

There are different size metrics : lines of code, number of bytes, number of modules. . . . The choice of measure is made according to the question to be answered (complexity, disk footprint,...)

LOC : Line Of Code → How to count them ? What about blank lines, comments, data declarations, several instructions on a line

- **NCLOC** : No Commented LOC. (No commented line, no blank line, everything else counts as 1 per line)
- **CLOC** : Commented LOC. (Line commented)

$$\Rightarrow \text{LOC} = \text{NCLOC} + \text{CLOC}$$

$$\Rightarrow \text{Comment Density} = \text{CLOC} / \text{LOC}$$

What count ? What files ?

1. Which files ?
 - ⇒ Program code, test drivers, automatically generated code, imported code
2. Which code ? the delivered or the developed ?
 - **Executable Statement (ES)** : No blank, no comments, no data nor headers, 1 per statement.
 - **Delivered Source Instruction (DSI)** : No blank, no comment, 1 per statement or data declaration

13.1.3 Explain functional size measurement using function points.

Function point (FP) : measure amount of functionality. Useful to :

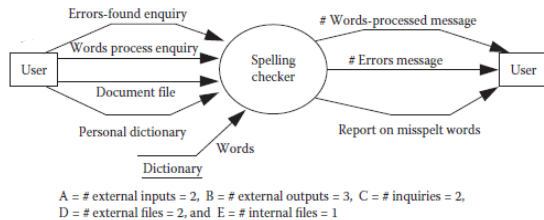
1. estimate the development effort and duration (mm/FP)
2. to express defect density (defects/FP)
3. to bill the development (\$/FP)

Example : Spell-checker spec

The checker accepts as input a document file and an optional personal dictionary file. The checker lists all words not contained in either of these files. The user can query the number of words processed and the number of spelling errors found at any stage during processing.

• **Point 1** : Number of items

- A : external files (file names, menu selection)
- B : external output (report message)
- C : external inquiries (interactive inputs requiring a response)
- D : external files (interfaces to other systems)
- E : internal files (master files in the system)



• **Point 2** : Complexity for each item (weight)
Unadjusted function points :

$$UFC = \sum (\#item\ of\ type\ i) * (weight\ of\ type\ i)$$

Item	Simple	Weighting Factor Average	Complex
External inputs	3	4	6
External outputs	4	5	7
External inquiries	3	4	6
External files	7	10	15
Internal files	5	7	10

Example : all average complexity

- A = 2, B = 3, C = 2, D = 2, E = 1
- $UFC = 4A + 5B + 4C + 10D + 7E = 58$

• **Point 3** : Technical complexity factors (TFC)
Each F_i is between 0 and 5 :

- $TFC = 0.65 + 0.01 \sum F_i$
- $FP = UFC * TFC$

F_1 Reliable backup and recovery	F_2 Data communications
F_3 Distributed functions	F_4 Performance
F_5 Heavily used configuration	F_6 Online data entry
F_7 Operational ease	F_8 Online update
F_9 Complex interface	F_{10} Complex processing
F_{11} Reusability	F_{12} Installation ease
F_{13} Multiple sites	F_{14} Facilitate change

Example : 6 factors at 0, 6 factors at 3, 2 factors at 5

- $TCF = 0.65 + 0.01 (6 \times 3 + 2 \times 5) = 0.93$
- $FP = 58 \times 0.93 = 54$

14 Software measurement : structure

14.1 Describe the principles of structure measurement. Describe hierarchical measures based on prime decomposition of control flow graphs. Define cyclomatic complexity measure. Discuss design-level, inter-modular complexity measures.

14.1.1 Describe the principles of structure measurement.

The **size** doesn't tell everything, the **structure** play a role very important too to capture the **complexity** of the product. The structure measurement is applicable to design or code seen as a graph.

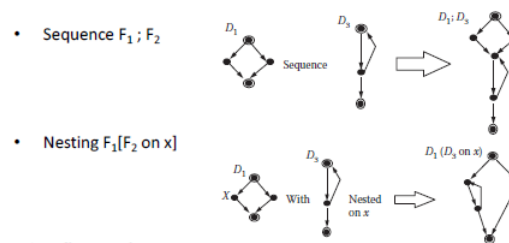
Two perspectives : Control Flow & Data Flow

Structural attributes :

1. **Complexity** : complicatedness of the connections between elements in a system model (positive, monotonic, additive on disjoint elements)
2. **Length** : distance between elements (positive, monotonic, max on disjoint elements)
3. **Coupling** : links to/from the elements outside the module (positive, monotonic, at most sum on merged modules)
4. **Cohesion** : connections between internal elements (0 to 1, monotonic, at most sum on merged modules)

14.1.2 Describe hierarchical measures based on prime decomposition of control flow graphs.

- **Control flow graph** : a graph with distinguished start and stop nodes. We want measures that are independent of a particular view (granularity) of the graph
- **Prime decomposition** : A structured flow graph can be decomposed into primes (Corresponds to a structured program construction). The decomposition is unique, can be done automatically and allows to check whether a program is S-structured (we decompose into primes and we check if all primes are in S) Given a family S of prime flowgraphs, a flow graph is S-structured iff it is generated from S by a finite number of sequencing and nesting.



- **Prime flow graph** : a graph that cannot be decomposed by sequencing or nesting

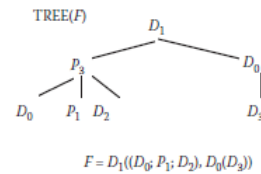
⇒ **Hierarchical measures** : measures defined on the prime decomposition tree. For example : depth of nesting.

More generally :

1. $M1 : M(F)$ for each F in S
 2. $M2 : L(F_1, \dots, F_n)$ from $M(F_i)$
 3. $M3 : M(F(F_1, \dots, F_n))$ from $M(F_i)$
- ⇒ any valid $M1, M2, M3$ gives a hierarchical measures (ex : size $v(F)$ number of nodes)

Example for size $v(F)$ (number of nodes)

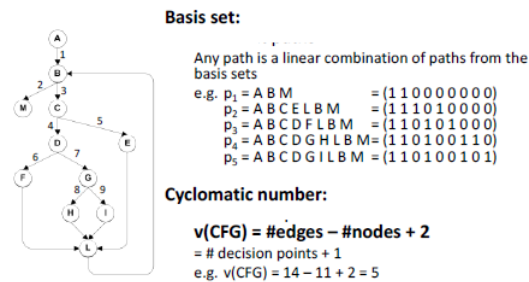
- M_1 : $v(P_1) = 1$, and for each prime $F \neq P_1$, $v(F) = n + 1$, where n is the number of procedure nodes in F
- M_2 : $v(F_1, \dots, F_n) = \sum v(F_i)$
- M_3 : $v(F(F_1, \dots, F_m)) = 1 + \sum v(F_i)$ for each prime $F \neq P_1$



14.1.3 Define cyclomatic complexity measure.

- **Basis set** : a maximal set of linearly independent paths (any path is a linear combination of paths from the basis set)
- **Cyclomatic number** : the number of path in a basis set.

$$v(CFG) = \#edges - \#nodes + 2 = \#decisionpoints + 1$$



- Cyclomatic complexity is hierarchical

M_1 : $v(F) = 1 + d$ for each prime F , where d is the number of predicates in F

M_2 : $v(F_1, \dots, F_n) = \sum_{i=1}^n v(F_i) - n + 1$ for each n

M_3 : $v(F(F_1, \dots, F_n)) = v(F) + \sum_{i=1}^n v(F_i) - n$ for each prime F

14.1.4 Discuss design-level, inter-modular complexity measures.

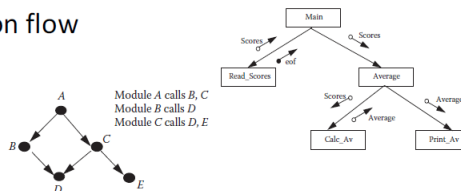
So far we have talked about intra-modular measures (inside a procedure). Now we will present inter-modular measures (dependencies between modules) . We thus consider design (module structure is the same for code)

Dependency graphs : graph representing the dependency between modules (information flow or call graph). Inside modules we have data dependency graphs.

We will consider **dependency graphs** between modules

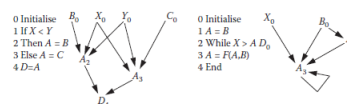
- information flow

- call graph



Inside modules:

data dependency graphs



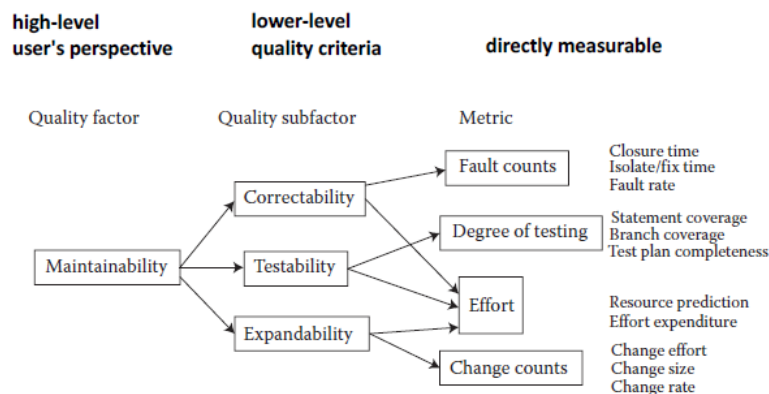
15 Software measurement : quality

15.1 Describe the principles and elements of quality models. Discuss defect-based quality measures. Discuss usability, maintainability and security measures.

15.1.1 Describe the principles and elements of quality models

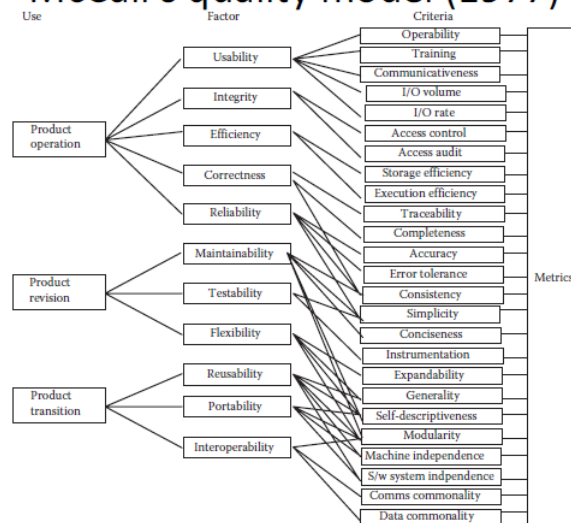
The question here is : Does a product have desirable attributes? (A product can be a document, a file, a system,... while an attribute can be completeness, consistency, reliability,...). Product quality models give a hierarchical nomenclature of product quality characteristics.

A quality models has different elements :

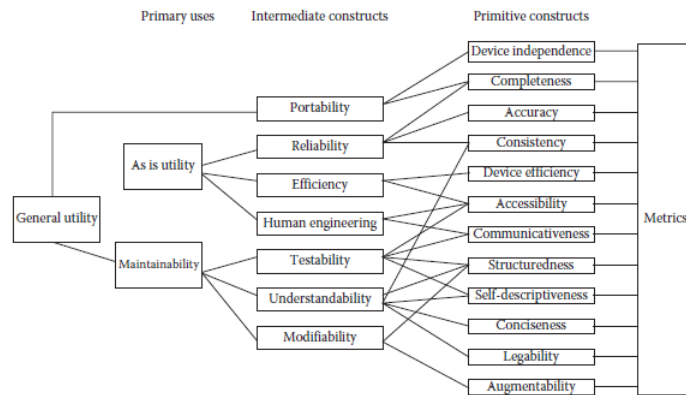


There are different models such like Boehm or McCall's quality model which are **fixed models**.

McCall's quality model (1977)



Boehm's quality model (1978)



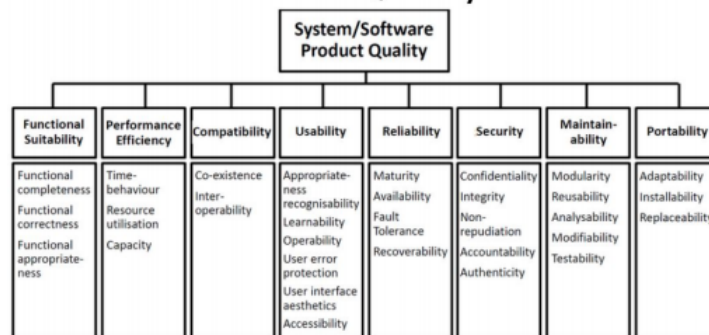
But we can also define our own models :

- Keep general philosophy of quality models (factors, criteria, metrics)
- Select attributes suited for a particular product
 - Discuss with customers, users
 - Possibly taken from some fixed model
- Refine to criteria, metrics

We can design it by measurable objectives :

- measurable attributes identified in the specification.
- Applicable to small, evolutionary development, agile processes 10 fixed model

ISO Standard Quality Models



15.1.2 Discuss defect-based quality measures.

We take a narrow view :

1. **Quality** = lack of defects
2. **Defects** = errors, faults, failures

There are known defects (from testing, inspection,...) and latent defect (unknown).

$$defect\ density = \frac{\#known\ defects}{product\ size}$$

From this equation, one need to define what counts as defects (faults, failures, post-test, post-release,...) and what counts as size (comments, data binaries, tests,...). It is also important to not confuse with defect rate which is relative to time and not size.

There are different type of defect-based measures, another example :

$$Spoilage = \frac{time\ to\ fix\ defects}{total\ dev\ time}$$

15.1.3 Discuss usability, maintainability and security measures.

Usability :

The degree to which a product or system can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use.

Example : User-friendliness (ease to learn, use, remember) or user satisfaction.

⇒ This is not a directly measurable attribute, so we need to define usability measurement

- **Effectiveness** : %of correctly completed tasks. User recall = remember information provided (task effectiveness= quantity x quality of tasks)
- **Efficiency** : time to complete a task, input rate
 - time efficiency = effectiveness / task time
 - productive period = productive time / task time
 - relative user efficiency = user efficiency / expert efficiency
- **Satisfactions** : questionnaires, biological measurements

Other usability measurement, not in ISO 25010 :

- **Accessibility** : disabilities (visual, hearing, physical)
- **Universality** : cultural norms, naming conventions
- **Trustfulness** : trust of users in the system (→ security)

Internal elements related to usability :

- good use of menus and graphics
- informative error messages
- help functions
- consistent interfaces
- well-structured manuals

and structure can also be measured. In particular, text structure affects readability and comprehensibility. But size and structure are poor measures of quality.

Maintainability :

The degree of effectiveness and efficiency with which a product or system can be modified by the intended maintainers. Easy to understand, enhance and correct. Maintenance can be :

- Corrective (fix bugs)
- Adaptive (changes, upgrades)
- Preventive (before failures occur)
- Perfective (enhance, extend)

They apply to code, documentation, specs, design, tests,... Maintenance is about making changes to the product. There are different maintainability measures :

- **(Most important one) Mean Time To Repair (MTTR)** : average time to implement a change and restore the system to working order, can be split in different measure times :
 - Problem recognition time
 - Administrative delay time
 - Maintenance tools collection time
 - Problem analysis time
 - Change specification time
 - Change time (including testing and review)
- Change time/number of changes
- Number of unresolved problems
- time spent on unresolved problems
- % changes that introduce new faults
- number of modules modified for a change

Concerning internal elements related to maintainability :

- **Structural complexity** : this is more an indication, not really a measure. Used in correlation with external measures (e.g. identify a module with poor structure)
- Readability : for texts. We can talk about the Fog Index

Security :

The degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization. No “competent programmer” hypothesis : assume that attackers try to overcome security protections and hide their activities.

Security measures :

1. **Risk** = Impact x Likelihood x Threat x Vulnerability (where impact, likelihood and vulnerability depend on threat).
2. **Common vulnerability Scoring System (CVSS)** give a metric between 0 and 1 based on 6 measures $CVSS = f(AV, AC, Au, C, I, A)$
 - (a) Access vector (AV) : how remote an attacker can be
 - (b) Access complexity (AC) : how complex the attack needs to be
 - (c) Authentication (Au) : how many authentications needed
 - (d) Confidentiality impact (C) : impact to system
 - (e) Integrity impact (I) : impact to system integrity
 - (f) Availability impact (A) : reduced performance, shutdown
3. **Bayesian analysis (probabilistic)** : based on attack probabilities and costs
4. #vulnerabilities / #defects

Internal measures : Number of entry points, exit points, data channels, persistent data items

16 Software reliability

16.1 Describe the model of reliability based on failure rates. Define failure probability functions, reliability, maintainability, availability, MTTF and MTTR. Describe the case of constant failure rates. Describe the principles of failure prediction systems. Discuss the example of the Jelinski-Moranda model. Discuss statistical testing.

16.1.1 Describe the model of reliability based on failure rates.

Reliability is a key quality attribute : top-level in all quality models and the most extensively studied. Reliability has as objectives :

1. Measure failures
 2. Predict future failures from past failures
 3. Reliability growth : predict reliability from past faults found and fixed
- ⇒ Want to answer a basic question : When will the system fail ?

Software failures are different from hardware failures. A component may fail due to physical wear. The probability that the component will fail at time t is given by the probability density function $f(t)$

16.1.2 Define failure probability functions, reliability, maintainability, availability, MTTF and MTTR.

- Failure probability **density function** is the probability that a failure will occur between time t and time $t+dt$:
 $f(t)dt = \text{Prob}(\text{failure between } t \text{ and } t+dt)$
- Failure probability **distribution function** is the probability that a failure will occur between 0 and t :
 $F(t) = \int_0^t f(t)dt = \text{Prob}(\text{failure between } 0 \text{ and } t)$
- **Reliability** : operating without failure for a given time interval. It is time-dependent :
 $R(t) = 1 - F(t) = \text{Prob}(\text{no failure before } t)$
- **Maintainability** : maintenance activity can be carried out within stated time interval, procedures and resources. Also time-dependent :
 $M(t) = \text{Prob}(\text{restored before } t)$
- **Availability** : operation without failure at a given point in time, time-independent.
 $A = \text{Prob}(\text{not failed}) = \text{MTTF} / (\text{MTTF} + \text{MTTR})$
- **MTTF** : Mean Time To Failure is the average time before failure occurs. It measures reliability.
- **MTTR** : Mean Time To Repair is the average time to fix a fault. It measures maintainability.

16.1.3 Describe the case of constant failure rates.

Constant failure rate : it is when the software will fail purely randomly independently of the past, no memory. We have a constant probability rate λ .

1. $f(t) = \lambda \exp(-\lambda t)$
2. $F(t) = 1 - \exp(-\lambda t)$
3. $R(t) = \exp(-\lambda t)$

16.1.4 Describe the principles of failure prediction systems.

Failure prediction systems Some basic notions first :

- Given a series of failure times ($t_1, t_2, \dots, t_{i-1}$) where the fault has been fixed after each failure. The goal of these systems is to predict future failure times $T_i, T_{i+1}, \dots$
- In software, as we fix faults, we expect the reliability to growth.
- The **goal of a prediction system** is to predict the probability distributions $F_i, F_{i+1}, \dots$
- A prediction system has three elements :
 1. **Prediction model** : probability specification of the stochastic process (distributions $F_i(T_i)$)
 2. **Inference procedure** : infer unknown parameters of the model from $t_1, t_2, \dots, t_{n-1}$
 3. **Prediction procedure** : combine the model and inference procedure to make predictions about future failure behavior

16.1.5 Discuss the example of the Jelinski-Moranda model.

Jelinski-Moranda model

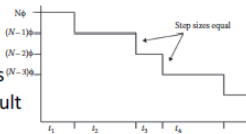
Model: constant rate ϕ identical for all faults

$$F_i(t_i) = 1 - \exp(-\lambda_i t_i)$$

$$\lambda_i = (N - i + 1) \phi$$

N = initial number of faults

ϕ = failure rate for each fault



Type-1 uncertainty = random, exponential rate

No type-2 uncertainty = corrections are perfect

Fixing any fault contributes equally to improving the reliability

Inference procedure: maximum likelihood estimation (not detailed)

Gives predictions for N_i and ϕ_i

Prediction procedure: predict mean time to next failure

$$T_i = 1/\lambda_i = 1 / (N_i - i + 1) \phi_i$$

Jelinski-Moranda model: example

TABLE 11.2 Successive Failure Times for JM when $N = 15$ and $\phi = 0.003$

i	Mean Time to i th Failure	Simulated Time to i th Failure
1	22	11
2	24	41
3	26	13
4	28	4
5	30	30
6	33	77
7	37	11
8	42	64
9	48	54
10	56	34
11	67	183
12	83	83
13	111	17
14	167	190
15	333	436

Jelinski-Moranda model: criticisms

Unrealistic assumptions:

- The sequence of λ_i is purely deterministic.
- All faults equally contribute to the hazard rate.

The **reliability predictions** obtained from the model are **poor**; they are usually **too optimistic**.

16.1.6 Discuss statistical testing.

Idea : reliability predictions based on failures occurring during testing. Will this be accurate in typical system usage? (With different users, tasks, experiences levels,...).

1. Operational Profile : probability distribution on inputs (reflecting usage)
2. Statistical testing : select tests according to operational profile
 - Tests focus on more used parts => better observed reliability
 - Test reflects usage => reliability predictions more accurate.

Operational profiles are difficult to define.

⇒ A small % of the operational profile may account for a large % of failures.

Examples :

1. Airplane : take-off and landing

2. Printer :

- non-saturated : available, no queue => print immediately (20%)
- saturated : busy, queue => add to queue (79%)
- transitional : busy, no queue => create queue and add to queue (1%)
- probability of failures : 0.001 per test case

To have a 50% chance of detecting each fault, me must run :

- non saturated : 2500 test cases
- saturated : 663 test cases
- transitional 50 000 test cases

Transitional likely the most complex and failure-prone !